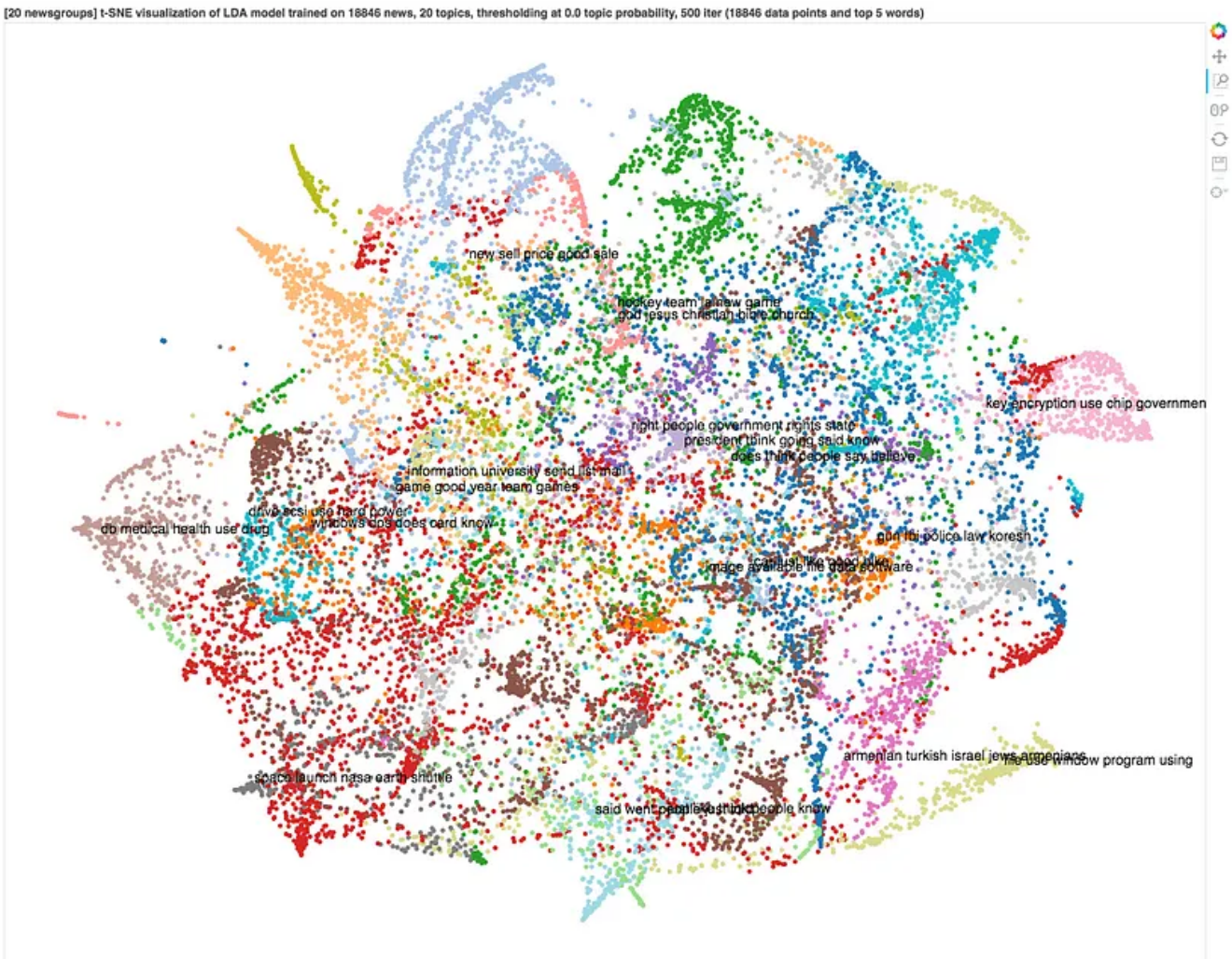
How Stuff Works: A Comprehensive Topic Modelling Guide with NMF, LSA, PLSA, LDA & lda2vec (Part-1)

Sourav Bose · Follow

12 min read · Dec 20, 2018

Listen Share

This article is a *comprehensive overview* of **Topic Modeling** and its associated techniques. This is the first part of the article and will cover NMF, LSA and PLSA only. The LDA and lda2vec will be covered in the next part here.



In natural language understanding (NLU) tasks, there is a hierarchy of lenses through which we can extract meaning — from words to sentences to paragraphs to documents. At the document level, one of the most useful ways to understand text is by analyzing its *topics*. The process of learning, recognizing, and extracting these topics across a collection of documents is called topic modeling.

In this post, we will explore topic modeling through 5 of the most popular techniques today: NMF, LSA, PLSA, LDA and the newer, deep learning-based lda2vec.

Overview

All topic models are based on the same basic assumption:

- each **document** consists of a mixture of ***topics***, and
- each ***topic*** consists of a collection of **words**.

NMF (Non-Negative Matrix Factorization)

With the rise of complex models like deep learning, we often forget simpler, yet powerful machine learning methods that can be equally insightful. NMF (Nonnegative Matrix Factorization) is one effective machine learning technique that I feel does not receive enough attention. NMF has a wide range of uses, from topic modeling to signal processing.

NMF (Nonnegative Matrix Factorization) is a matrix factorization method where we constrain the matrices to be nonnegative. In order to understand NMF, we should clarify the underlying intuition between matrix factorization.

Suppose we factorize a matrix X into two matrices W and H such that

$$X \approx WH$$

There is no guarantee that we can recover the original matrix, so we will approximate it as best as we can.

Now, suppose that X is composed of m rows,

And W is composed of k rows,

$$w_1, w_2, \ldots w_k$$

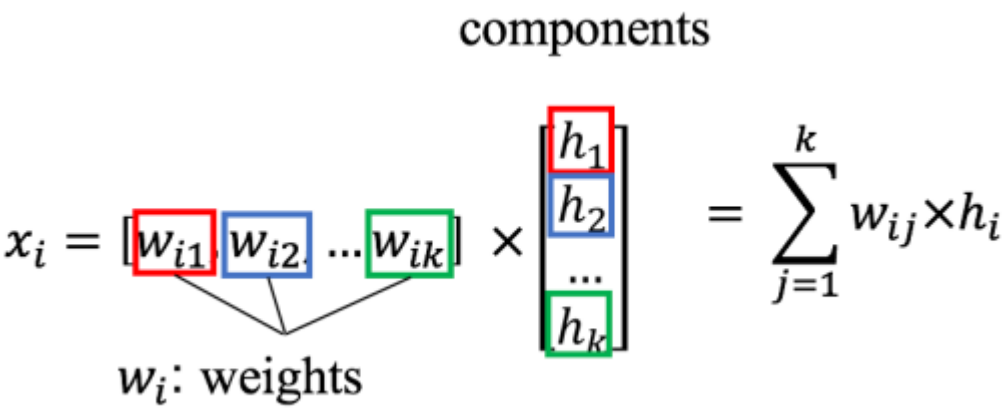
H is composed of m rows.

$$h_1, h_2, \ldots h_m$$

Each row in X can be considered a data point where m may/may not be equal to k. For instance, in the case of decomposing images, each row in X is a single image, and each column represents some feature.

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \ldots \\ x_k \end{bmatrix} \quad W = \begin{bmatrix} w_1 \\ w_2 \\ \ldots \\ w_k \end{bmatrix} \quad H = \begin{bmatrix} h_1 \\ h_2 \\ \ldots \\ h_k \end{bmatrix}$$

The meaning of this equation becomes clearer when we visualize it:



Basically, we can interpret

$$x_i$$

to be a weighted sum of some components (or bases if you are more familiar with linear algebra), where each row in H is a component, and each row in W contains the weights of each component.

Note that in this explanation, we treat each *row* of the input matrix X to be a single data point. In other explanations, each *column* might be considered a data point, in which case each column in W becomes a component and each column in H becomes a set of weights.

In practice, we introduce various conditions on the components, so that they can be interpreted in a meaningful manner. In the case of NMF, we constrict the underlying components and weights to be non-negative. Essentially, NMF decomposes each data point into an overlay of certain components.

When do we use NMF?

NMF is suited for tasks where the underlying factors can be interpreted as non-negative. Imagine if you wanted to decompose a term-document matrix, where each column represented a document, and each element in the document represented the weight of a certain word (the weight might be the raw count or the tf-idf weighted count or some other encoding scheme; those details are not important here).

What happens when we decompose this into two matrices? Imagine if the documents came from news articles. The word “eat” would be likely to appear in food-related articles, and therefore co-occur with words like “tasty” and “food”. Therefore, these words would probably be grouped together into a “food” component vector, and each article would have a certain weight of the “food” topic.

Therefore, an NMF decomposition of the term-document matrix would yield

Note that this interpretation would not be possible with other decomposition methods. We cannot interpret what it means to have a “negative” weight of the food topic. This is another example where the underlying components (topics) and their weights should be non-negative.

Another interesting property of NMF is that it naturally produces sparse representations. This makes sense in the case of topic modeling: documents generally do not contain a large number of topics.

How do we conduct NMF?

In order to conduct NMF we formalize an objective function and iteratively optimize it. NMF is an NP-hard problem in general, so we will aim for a good local minima.

Let's look at the objective function. Although there are some variants, a generally used measures of distance is the frobenius norm (the sum of element-wise squared errors). Formalizing this, we obtain the following objective:

$$minimize \, \|X - WH\|_F^2 \, w.r.t. \, W, H \, s.t. \, W, H \geq 0$$

Next, we will introduce a couple of optimization methods.

1. Multiplicative Update:

A commonly used method of optimization is the multiplicative update method. In this method, W and H are each updated iteratively according to the following rule:

$$H \leftarrow H \odot \frac{W^T X}{W^T W H}$$

$$W \leftarrow W \odot \frac{X H^T}{W H H^T}$$

where the matrix not being updated is kept constant.

The objective functions can be proved to be non-increasing under this rule. A less formal, but more intuitive explanation of this method is to interpret this as rescaled gradient descent.

Rescaled gradient descent can be written as follows:

$$H \leftarrow H - \eta \odot [W^T W H - W^T X]$$

$$W \leftarrow W - \zeta \odot [W H H^T - X H^T]$$

If we set

$$\eta$$

to be

$$\frac{H}{W^T W H}$$

and

$$\zeta$$

to be

$$\frac{W}{W H H^T}$$

then we obtain the multiplicative update rule.

2. Hierarchical Alternating Least Squares

Though less commonly used, in this paper, Hierarchical Alternating Least Squares (HALS) is shown to be a faster method for computing NMF.

HALS updates one column of W at a time, using the property that the columns of W do not interact with each other. It computes the closed-form optimal solution for each column, keeping all other columns constant.

The rule can be expressed as follows:

$$W[:,i] \leftarrow max(0, \frac{X h_i^T - \sum_{k \neq i} W[:,i] h_k h_i^T}{\|h_i\|^2})$$

where W[:, i] is the i-th column of W and hi is the i-th row of H.

Other commonly used tricks

1. **Initialization**
   Any iterative optimization scheme is sensitive to its initialization. Though random initialization often performs sufficiently well, other methods of initialization also exist.
   One such method uses SVD to compute a rough estimate of the matrices W and H. If the non-negativity condition did not exist, taking the top k singular values and their corresponding vectors would construct the best rank k estimate, measured by the frobenius norm. Since W and H must be non-negative, we must slightly modify the vectors we use. Detailed discussion is beyond the scope of this blog post, so please refer to this paper for the details.

2. **Regularization**
   We discussed the unregularized objective above, but we may want to impose stronger sparsity constraints or prevent the weights from becoming too large. To solve these problems, we can introduce L1 and L2 regularization losses on the weights of the matrices.

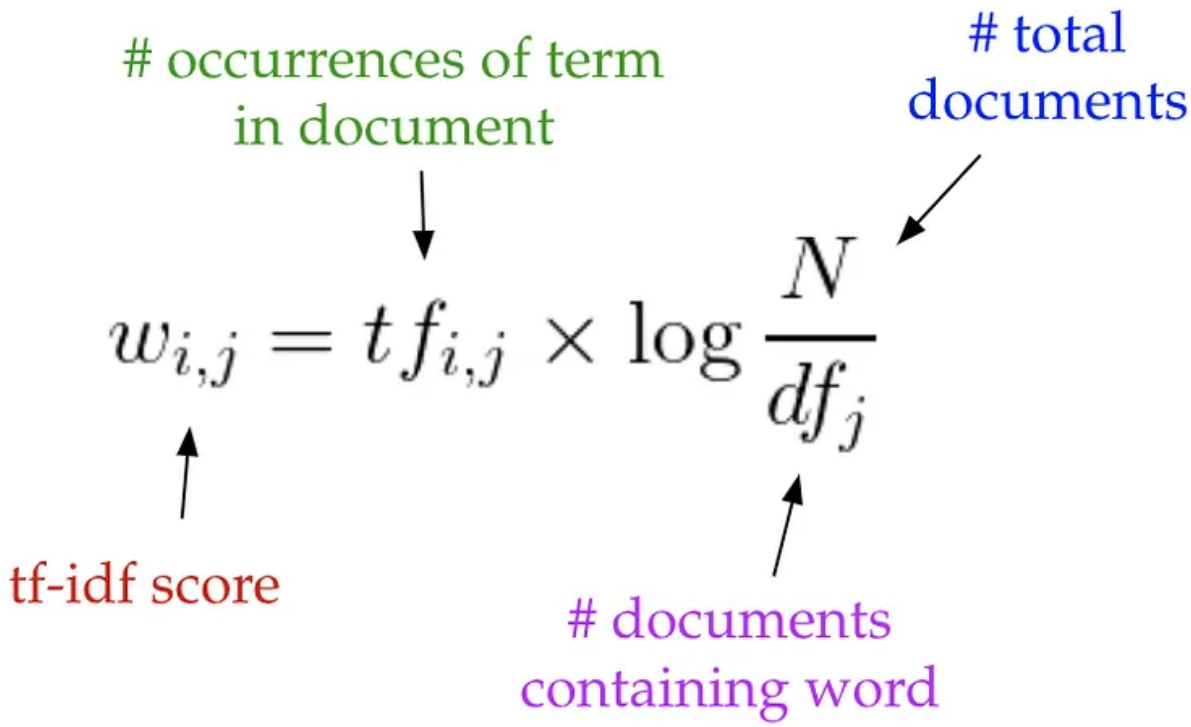
LSA (Latent Semantic Analysis)

Latent Semantic Analysis, or LSA, is one of the foundational techniques in topic modeling. The core idea is to take a matrix of what we have — documents and terms — and decompose it into a separate document-topic matrix and a topic-term matrix.

Latent Semantic Analysis, or LSA, is one of the foundational techniques in topic modeling. The core idea is to take a matrix of what we have — documents and terms — and decompose it into a separate document-topic matrix and a topic-term matrix.

The first step is generating our document-term matrix. Given m documents and n words in our vocabulary, we can construct an $m \times n$ matrix A in which each row represents a document and each column represents a word. In the simplest version of LSA, each entry can simply be a raw count of the number of times the j-th word appeared in the i-th document. In practice, however, raw counts do not work particularly well because they do not account for the *significance* of each word in the document. For example, the word “nuclear” probably informs us more about the topic(s) of a given document than the word “test.”

Consequently, LSA models typically replace raw counts in the document-term matrix with a **tf-idf score**. Tf-idf, or term frequency-inverse document frequency, assigns a weight for term j in document i as follows:



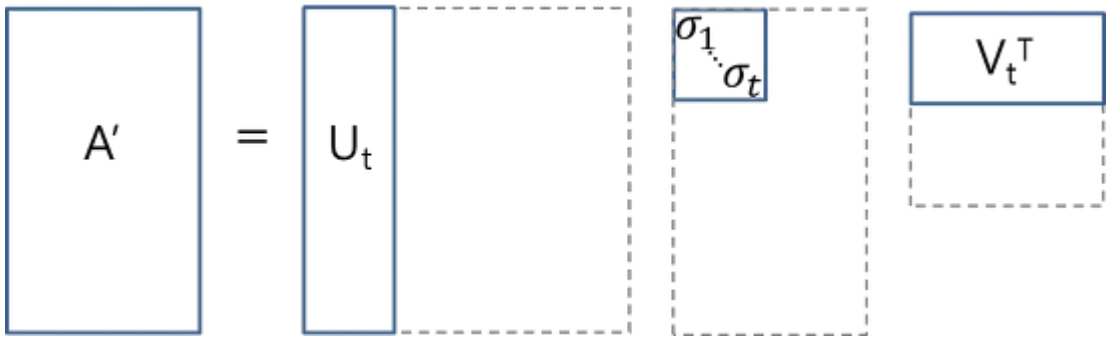
Intuitively, a term has a large weight when it occurs frequently across the *document* but infrequently across the *corpus*. The word "build" might appear often in a document, but because it's likely fairly common in the rest of the corpus, it will not have a high tf-idf score. However, if the word "gentrification" appears often in a document, because it is rarer in the rest of the corpus, it will have a higher tf-idf score.

Once we have our document-term matrix *A*, we can start thinking about our latent *topics*. Here's the thing: in all likelihood, *A* is very sparse, very noisy, and very redundant across its many dimensions. As a result, to find the few latent topics that capture the relationships among the words and documents, we want to perform dimensionality reduction on *A*.

This dimensionality reduction can be performed using **truncated SVD**. SVD, or singular value decomposition, is a technique in linear algebra that factorizes any matrix *M* into the product of 3 separate matrices: *M=U*S*V*, where *S* is a diagonal matrix of the singular values of *M*. Critically, truncated SVD reduces dimensionality by selecting only the *t* largest singular values, and only keeping the first *t* columns of *U* and *V*. In this case, *t* is a hyperparameter we can select and adjust to reflect the number of topics we want to find.

$$A \approx U_t S_t V_t^T$$

Intuitively, think of this as only keeping the *t* most significant dimensions in our transformed space.



In this case, $U \in \mathbb{R}^{(m \times t)}$ emerges as our document-topic matrix, and $V \in \mathbb{R}^{(n \times t)}$ becomes our term-topic matrix. In both *U* and *V*, the columns correspond to one of our *t* topics. In *U,* rows represent document vectors expressed in terms of topics; in *V,* rows represent term vectors expressed in terms of topics.

With these document vectors and term vectors, we can now easily apply measures such as cosine similarity to evaluate:

- the similarity of different documents
- the similarity of different words
- the similarity of terms (or "queries") and documents (which becomes useful in information retrieval, when we want to retrieve passages most relevant to our search query).

Code

In sklearn, a simple implementation of LSA might look something like this:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.pipeline import Pipeline

documents = ["doc1.txt", "doc2.txt", "doc3.txt"]

# raw documents to tf-idf matrix:

vectorizer = TfidfVectorizer(stop_words='english',
                             use_idf=True,
                             smooth_idf=True)

# SVD to reduce dimensionality:
```

```
svd_model = TruncatedSVD(n_components=100,         // num dimensions
                         algorithm='randomized',
                         n_iter=10)

# pipeline of tf-idf + SVD, fit to and applied to documents:

svd_transformer = Pipeline([('tfidf', vectorizer),
                            ('svd', svd_model)])

svd_matrix = svd_transformer.fit_transform(documents)

# svd_matrix can later be used to compare documents, compare words,
or compare queries with documents
```

LSA is quick and efficient to use, but it does have a few primary drawbacks:

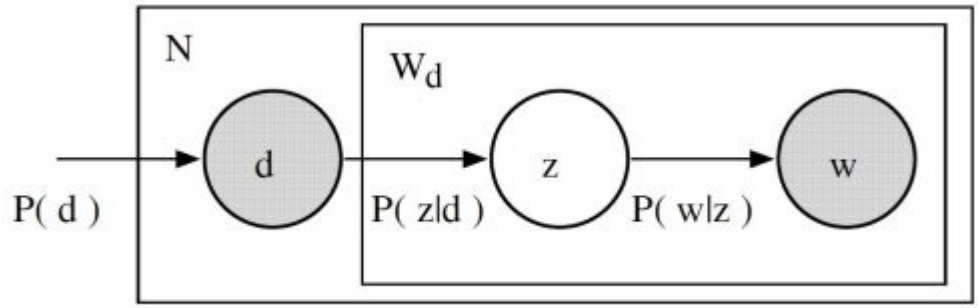
- lack of interpretable embeddings (we don't know what the topics are, and the components may be arbitrarily positive/negative)
- need for *really* large set of documents and vocabulary to get accurate results
- less efficient representation

PLSA (Probabilistic latent Semantic Analysis)

PLSA, or Probabilistic Latent Semantic Analysis, uses a probabilistic method instead of SVD to tackle the problem. The core idea is to find a probabilistic model with latent topics that can *generate* the data we observe in our document-term matrix. In particular, we want a model P(D,W) such that for any document *d* and word *w,* P(*d,w*) corresponds to that entry in the document-term matrix.

Recall the basic assumption of topic models: each document consists of a mixture of topics, and each topic consists of a collection of words. pLSA adds a probabilistic spin to these assumptions:

- given a document d, topic z is present in that document with probability P(z|d)
- given a topic z, word w is drawn from z with probability P(w|z)



Formally, the joint probability of seeing a given document and word together is:

$$P(D,W) = P(D)\sum_{Z} P(Z|D)P(W|Z)$$

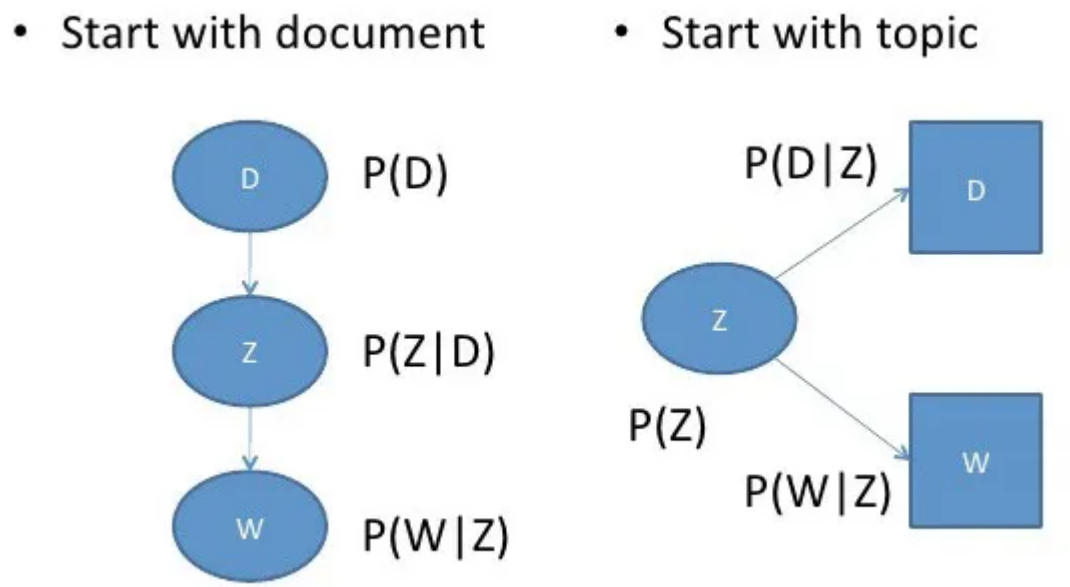
Intuitively, the right-hand side of this equation is telling us *how likely it is see some document,* and then based upon the distribution of topics of that document, *how likely it is to find a certain word within that document*.

In this case, P(D), P(Z|D), and P(W|Z) are the parameters of our model. P(D) can be determined directly from our corpus. P(Z|D) and P(W|Z) are modeled as multinomial distributions, and can be trained using the expectation-maximization algorithm (EM). Without going into a full mathematical treatment of the algorithm, EM is a method of finding the likeliest parameter estimates for a model which depends on unobserved, latent variables (in our case, the topics).

Interestingly, P(D,W) can be equivalently parameterized using a different set of 3 parameters:

$$P(D,W) = \sum_{Z} P(Z)P(D|Z)P(W|Z)$$

We can understand this equivalency by looking at the model as a generative process. In our first parameterization, we were starting with the document with P(d), and then generating the topic with P(z|d), and then generating the word with P(w|z). In *this* parameterization, we are starting with the topic with P(z), and then independently generating the document with P(d|z) and the word with P(w|z).



The reason this new parameterization is so interesting is because we can see a direct parallel between our pLSA model our LSA model:

$$P(D,W) = \sum_Z P(Z)P(D|Z)P(W|Z)$$

$$A \approx U_t S_t V_t^T$$

where the probability of our topic P(Z) corresponds to the diagonal matrix of our singular topic probabilities, the probability of our document given the topic P(D|Z) corresponds to our document-topic matrix *U*, and the probability of our word given the topic P(W|Z) corresponds to our term-topic matrix *V*.

So what does that tell us? Although it looks quite different and approaches the problem in a very different way, pLSA really just adds a probabilistic treatment of topics and words on top of LSA. It is a far more flexible model, but still has a few problems. In particular:

- Because we have no parameters to model P(D), we don't know how to assign probabilities to new documents
- The number of parameters for pLSA grows linearly with the number of documents we have, so it is prone to overfitting

We will not look at any code for PLSA because it is rarely used on its own. In general, when people are looking for a topic model beyond the baseline performance LSA gives, they turn to LDA. LDA, the most common type of topic model, extends PLSA to address these issues.

Please continue reading the next part of the article here.

Topic Modeling | Latent Semantic Indexing | Sklearn | Machine Learning | Natural Language Process

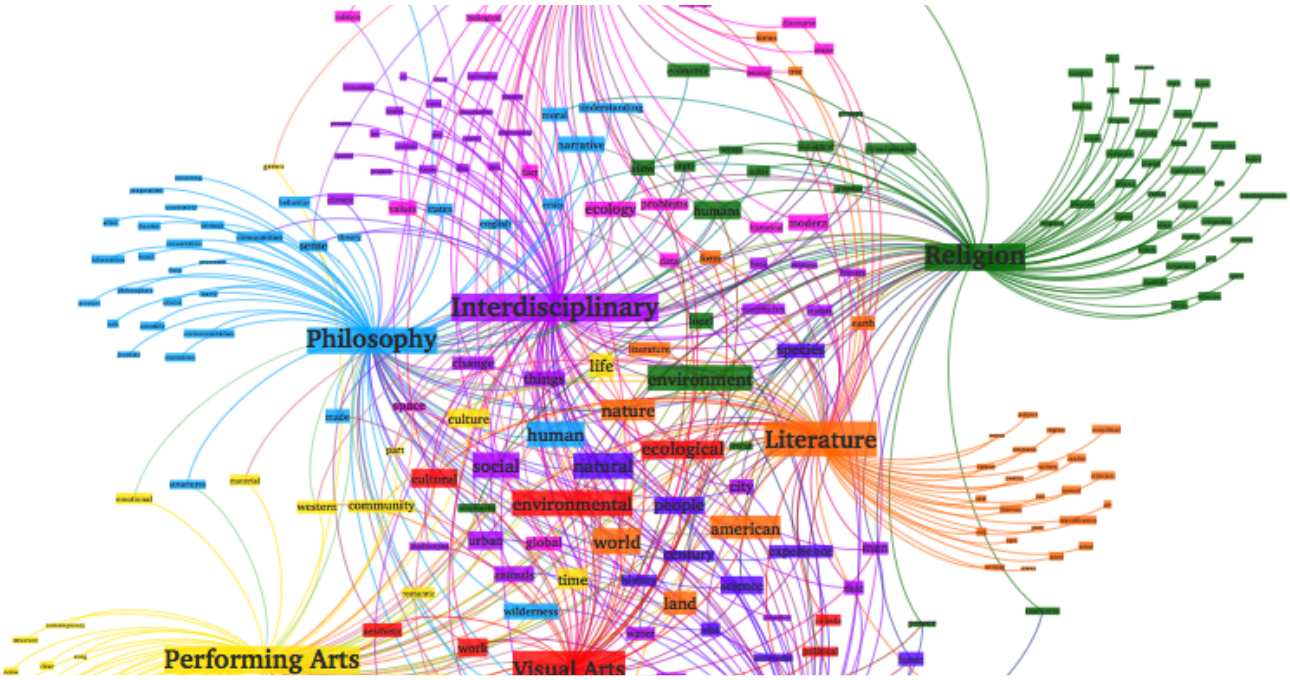
Follow

Written by Sourav Bose

41 Followers

Artificial Intelligence | Deep Learning | Machine Learning Professional who enjoys teaching and also happens to be a Cat-lover

More from Sourav Bose



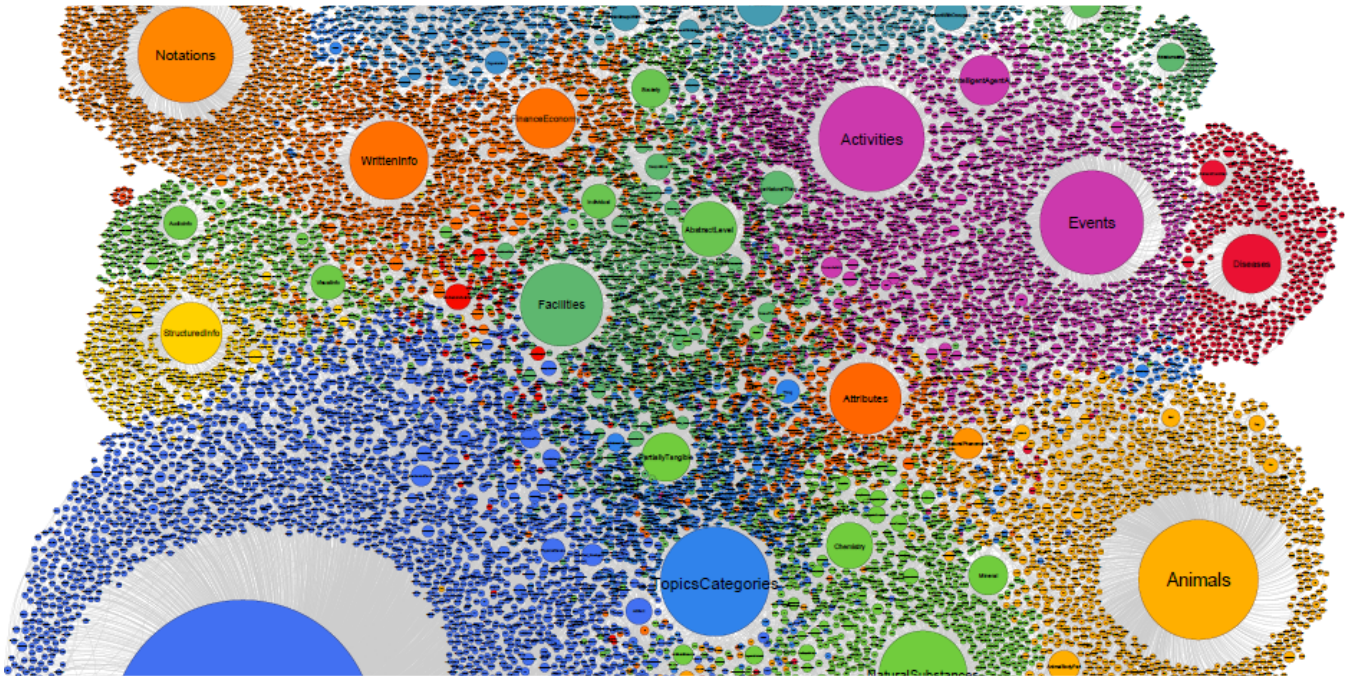
Sourav Bose

Comprehensive Topic Modelling with NMF, LSA, PLSA, LDA & lda2vec (Part-2)

This article is a comprehensive overview of Topic Modeling and its associated techniques. This is the second part of the article and will...

15 min read · Sep 3, 2019

--



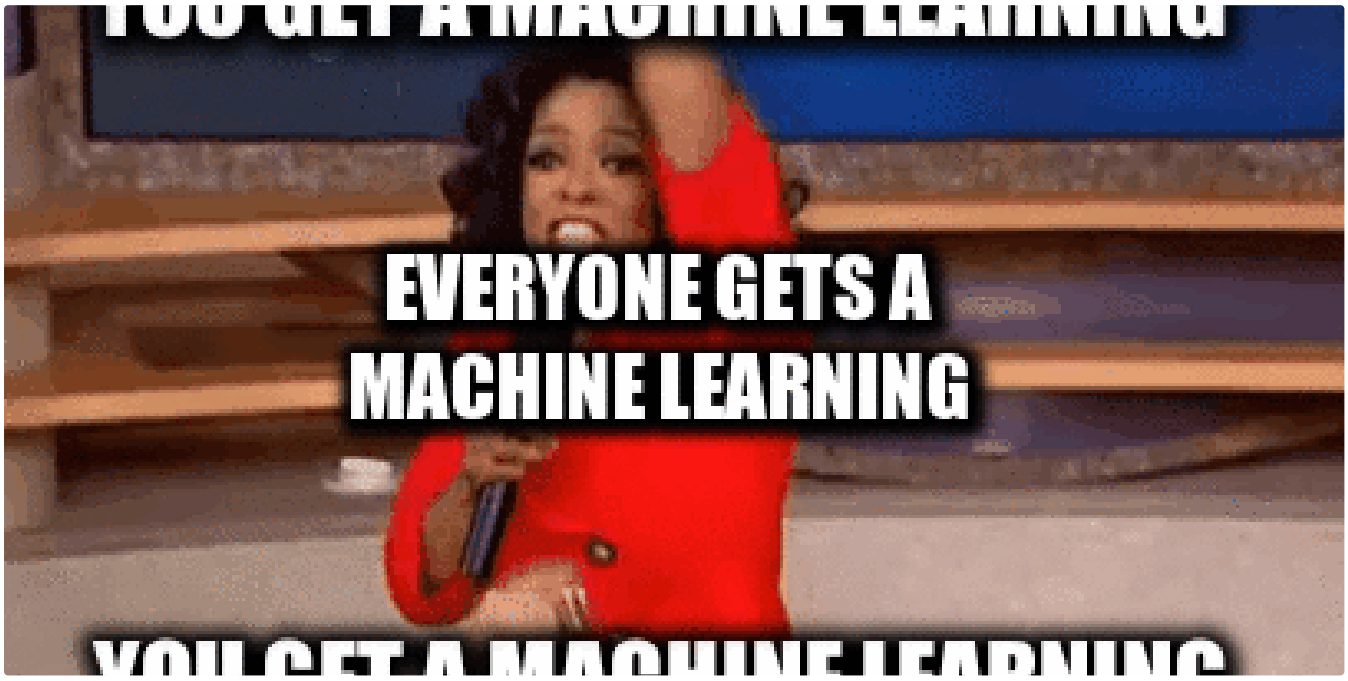
Sourav Bose

How Stuff Works: K-Means Clustering

Overview

22 min read · Aug 19, 2019

--



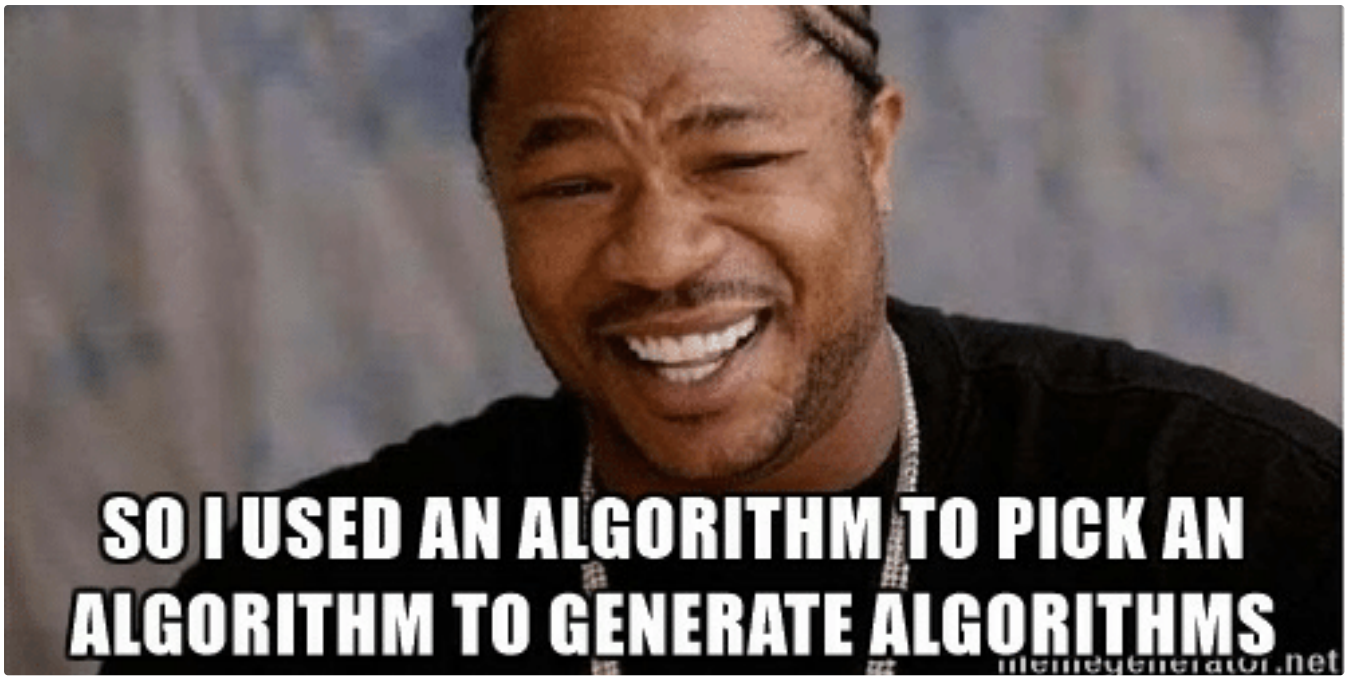
Sourav Bose

Introduction to Machine Learning

In this article am going to introduce to you what these buzzwords like Machine Learning means in simplistic terms and talks about the brad…

8 min read · Sep 3, 2019

--



Sourav Bose

Gentle Overview of major Machine Learning Algorithms

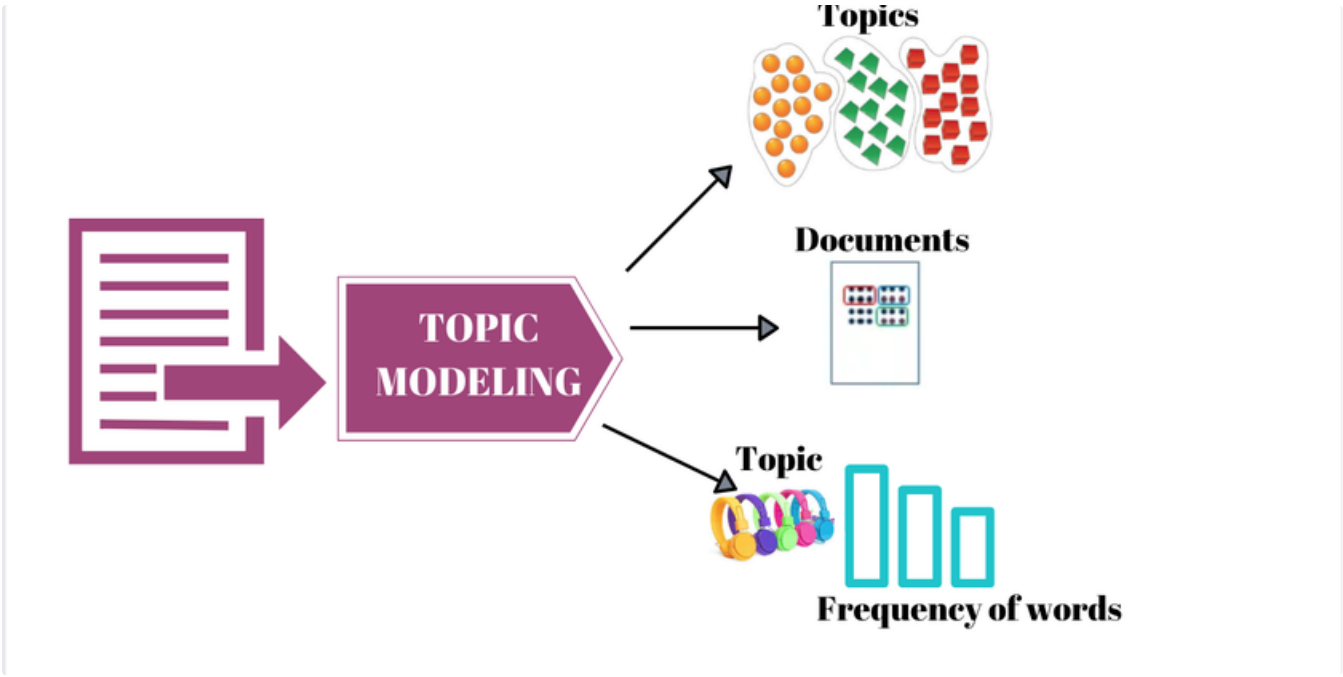
By this time I am assuming that you have a decent idea of what is Machine Learning and what are the broad types of its application and...

6 min read · Sep 3, 2019

--

See all from Sourav Bose

Recommended from Medium



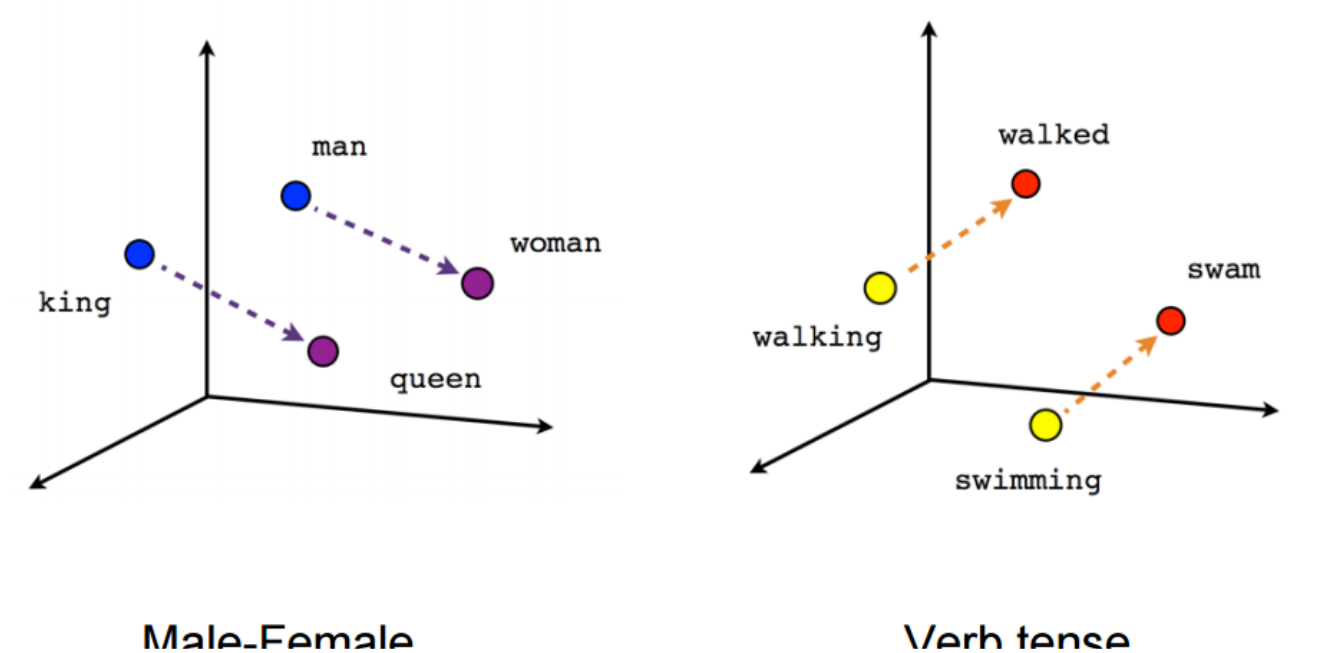
Zaheer

Topic Modeling with LDA

Hi everyone, welcome to my blog! Today I'm going to talk about a very interesting and useful topic in natural language processing (NLP)...

7 min read · May 3

-- 2



Male-Female Verb tense

Maninder Singh

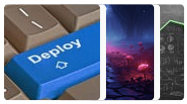
Accelerate Your Text Data Analysis with Custom BERT Word Embeddings and TensorFlow

One thing is for sure the way humans interact with each other naturally is one of the most difficult tasks for computers to understand…

4 min read · Apr 24

--

Lists

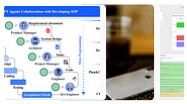


Predictive Modeling w/ Python

20 stories · 450 saves

Practical Guides to Machine Learning

10 stories · 517 saves



Natural Language Processing

669 stories · 283 saves

The New Chatbots: ChatGPT, Bard, and Beyond

13 stories · 133 saves

Kalyanchilakamarri

Integrating ChatGPT and Topic Modeling

Hi Medium readers hope you all are doing well, what's the hot topic in the town? It's ChatGPT. Although AI implementation was quite evident...

2 min read · May 22

--

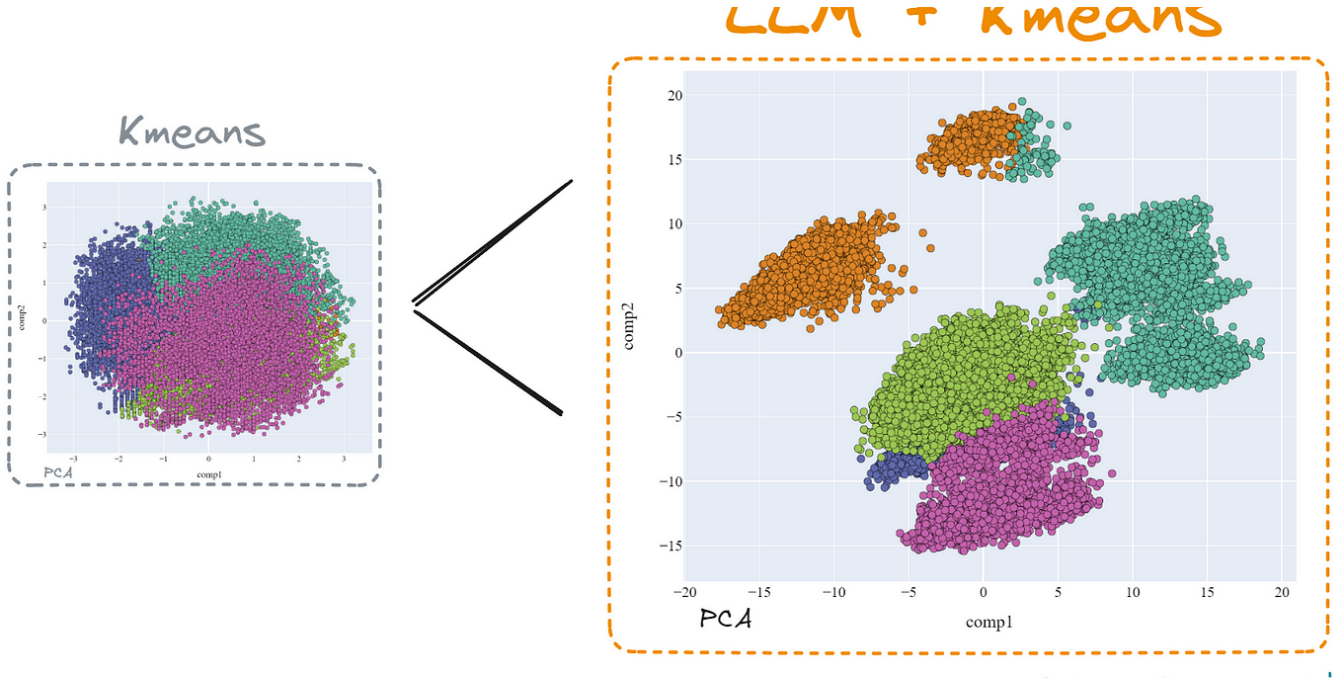
Abish Pius in Writing in the World of Artificial Intelligence

Analyzing Survey Results with LDA Topic Modeling

Preface

6 min read · May 6

--



Damian Gil in Towards Data Science

Mastering Customer Segmentation with LLM

Unlock advanced customer segmentation techniques using LLMs, and improve your clustering models with advanced techniques

23 min read · Sep 26

-- 21

Jawwad Shadman Siddique

Topic Modeling Using BERTopic on Newsgroup Dataset: Python Implementation

We go step by step from creating a google collab workspace to visualizing the cluster of topics in a group of text documents. The...

7 min read · Jul 4

 -- 

See more recommendations